

Имею много вставок типа `insertBefore`. Но страница может перегружаться (например в Blogger) и это приведет к замусориванию,- правда? Хочу в С дать свою функцию `InsertBefore` которая будет контролировать вставки и даст возможность потом их все поудалять. Так можно? или посоветуешь что-то более разумное?

Коротко и честно: **да, опасение абсолютно реальное**, и твоя идея с централизованным `InsertBefore` — **здвая**. Но её можно сделать **проще, надёжнее и дешевле**, чем «перехватывать все `insertBefore`».

Разложу по полочкам.

Что реально происходит при SPA / Blogger

В Blogger, Turbo, PJAX, SPA и т.п.:

- **DOM частично перерисовывается**
- старые узлы могут:
 - быть удалены без твоего ведома
 - или **остаться**, но страница уже «другая»
- твои `insertBefore`:
 - **не откатываются**
 - **не отслеживаются**
 - **могут дублироваться**

☞ Итог: **да, замусоривание возможно**
(дубли, утечки ссылок, висящие обработчики)

✗ Перехватывать `Node.prototype.insertBefore` — плохая идея

Теоретически можно, **практически не надо**:

- ломаешь инварианты браузера
- рискуешь чужими библиотеками
- сложно дебажить
- сложно восстановить порядок

Это путь боли.

✓ Правильный подход: «всё своё — в одном контейнере»

Золотое правило SPA-скриптов:

☞ Всё, что ты добавил — должно быть легко удалить одним движением.

Самый разумный вариант

Создай **корневой контейнер** для своих вставок:

```
function getRoot() {
  let root = document.getElementById('o5-root')
  if (!root) {
    root = document.createElement('div')
    root.id = 'o5-root'
    root.style.display = 'contents' // важно!
    document.body.appendChild(root)
  }
  return root
}
```

И дальше:

```
getRoot().insertBefore(node, refNode)
```

Удаление:

```
getRoot().remove()
```

- ✓ удаляется всё сразу
- ✓ не нужно отслеживать каждую вставку
- ✓ `display: contents` не ломает верстку

⚠ Когда `display: contents` нельзя

Если браузер старый или есть проблемы со стилями — тогда:

```
root.dataset.o5 = 'root'
```

И стили максимально нейтральные.

✓ Альтернатива: «реестр вставок» (ближе к твоей идее)

Если контейнер не подходит (вставляешь в разные места DOM):

```
C.dom = {
  inserted: new Set(),

  insertBefore(node, ref) {
    ref.parentNode.insertBefore(node, ref)
  }
}
```

```

        this.inserted.add(node)
        return node
    },
    cleanup() {
        for (const node of this.inserted) {
            if (node.isConnected)
                node.remove()
        }
        this.inserted.clear()
    }
}

```

Вызывать при смене страницы:

`C.dom.cleanup()`

- ✓ работает
 - ✓ прозрачно
 - ✗ требует дисциплины (везде использовать только этот метод)
-

✓ Лучший вариант (если хочешь «по уму»)

Маркировать, а не регистрировать

`node.dataset.o5 = '1'`

Очистка:

`document.querySelectorAll('[data-o5]').forEach(n => n.remove())`

Плюсы:

- не важно, как вставили
- не важно, потерял ли ты ссылку
- работает даже после частичной перерисовки

☞ Это самый живучий вариант для Blogger.

! Отдельно про обработчики событий

Даже если DOM очищен — утечки могут быть из-за:

- `addEventListener` на `window/document`
- `setInterval`
- `MutationObserver`

Ты уже идёшь правильным путём с `close()` модулей 👍

Туда же:

- `observer.disconnect()`
 - `clearInterval`
 - `removeEventListener`
-

Рекомендация без украшательств

Если кратко:

❶ Лучшее всего:

→ маркировать свои элементы `data-o5` и чистить по селектору

❷ Если нужна структура:

→ один `o5-root` контейнер

❸ Реестр `InsertBefore` — допустимо, но не идеально

❹ Перехватывать нативный `insertBefore` — ✗ не надо

Если хочешь, могу:

- предложить **готовый C . DOM** модуль под твою архитектуру
- или показать **как привязать очистку к твоему `Page.close()`**
- или разобрать конкретный кейс Blogger (он коварный)

Скажи, в какую сторону копаем дальше.

покажи как привязать очистку к `Page.close()`

Отлично, показываю **прямо под твою архитектуру**, без абстракций и «фреймворкового тумана».

Ниже — **минимальный, надёжный вариант**, который:

- не трогает нативный DOM
 - переживает Blogger / SPA
 - чистится **одной строкой в `Page.close()`**
 - не требует помнить ссылки на узлы
-

1❑ Модуль C . DOM (реестр + маркер)

```
// C.DOM.js
export function initDOM(C) {

    const MARK = 'o5'

    C.DOM = {
        mark(node) {
            if (node && node.nodeType === 1)
                node.dataset[MARK] = '1'
            return node
        },

        insertBefore(node, ref) {
            this.mark(node)
            ref.parentNode.insertBefore(node, ref)
            return node
        },

        append(parent, node) {
            this.mark(node)
            parent.appendChild(node)
            return node
        },

        cleanup() {
            document
                .querySelectorAll(`[data-${MARK}]`)
                .forEach(n => n.remove())
        }
    }
}
```

📖 Обрати внимание:

мы **не храним ссылки вообще**, только маркер в DOM.

2❑ Использование в модулях (пример)

```
const panel = document.createElement('div')
panel.textContent = 'Моя панель'

C.DOM.insertBefore(panel, document.body.firstChild)

или

C.DOM.append(document.body, panel)
```

3❑ Привязка к Page . close () (ключевой момент)

В твоём классе Page **ничего сложного не нужно**:

```
close() {
  clearInterval(this.timer)

  if (C.DOM)
    C.DOM.cleanup()

  for (const name in C.modules) {
    const mod = C.modules[name].mod
    if (mod && mod.close)
      mod.close()
  }
}
```

- ✓ очищается **всё добавленное**
- ✓ не важно, сколько раз страница перезагружалась
- ✓ не важно, кто и где вставлял

4☐ Когда вызывать Page.close()

Ты уже это делаешь правильно:

```
if (this.hasBeenInitialized)
  this.close()
```

☞ именно **до** новой инициализации страницы.

5☐ Почему это решение живучее

Проблема	Решение
SPA / Blogger	селектор переживает DOM-хаос
Потеря ссылок	не храним ссылки
Дубли	всё чистится
Ошибки вставок	не важны
Производительность	querySelectorAll дешёвый

6☐ Хочешь усилить (не обязательно)

Очистка только внутри body

```
document.body.querySelectorAll('[data-o5]')
```

Или только своего root

```
#o5-root [data-o5]
```

7 Маленький бонус: защита от двойной вставки

```
if (document.querySelector('[data-o5-panel]')) return
```

Итог — без лирики

- ✓ Привязка к `Page.close()` — одна строка
- ✓ Работает в Blogger
- ✓ Не ломает DOM
- ✓ Масштабируется

Если хочешь, следующим шагом могу:

- сделать **типизированный JSDoc** для `C.DOM`
- или встроить это в твой `inc`-модуль
- или добавить **`MutationObserver`** для автозачистки

Говори, что дальше.